

Aula 2 - Metodologias Ágeis e Qualidade (Scrum + CI/CD) com Prática em Java

Duração: 2 horas

Objetivos:

- Entender como metodologias ágeis (Scrum, Kanban) impactam a qualidade de software.
- Introduzir conceitos de Integração Contínua (CI) e Entrega Contínua (CD).
- Praticar CI/CD básico com GitHub Actions em um projeto Java simples.

Parte Teórica (1h)

1. Metodologias Ágeis e Qualidade

Scrum:

- **Papéis:** Product Owner, Scrum Master, Dev Team.
- **Artefatos:** Product Backlog, Sprint Backlog, Incremento.
- **Eventos:** Daily, Sprint Planning, Review, Retrospective.

Kanban:

- Visualização do fluxo (quadro Kanban).
- Limite de trabalho em progresso (WIP).

Como isso melhora a qualidade?

- ✓ **Feedback constante** (revisões frequentes).
- ✓ **Entrega incremental** (detecção precoce de bugs).
- ✓ **Transparência** (todos veem o progresso).

2. Integração Contínua (CI) e Entrega Contínua (CD)

- **CI:** Automatizar build e testes a cada commit.
- **CD:** Automatizar deploy em ambientes de teste/produção.
- **Ferramentas:** GitHub Actions, Jenkins, GitLab CI.

Benefícios para Qualidade:

- Reduz "integration hell".

- Garante que o código sempre passe nos testes.

Parte Prática (1h) - CI/CD com GitHub Actions

Atividade: Configurar um Pipeline CI Básico para Projeto Java

Passo 1: Criar um Projeto Java Simples

- Projeto Maven/Gradle com um teste JUnit (exemplo abaixo).

src/main/java/Calculator.java

```
java

public class Calculator {
    public static int add(int a, int b) {
        return a + b;
    }
}
```

src/test/java/CalculatorTest.java

```
java

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class CalculatorTest {
    @Test
    public void testAdd() {
        assertEquals(4, Calculator.add(2, 2));
    }
}
```

Passo 2: Configurar GitHub Actions (CI)

1. Criar arquivo `.github/workflows/ci.yml` no repositório:

```
yaml

name: Java CI

on: [push, pull_request]
```

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Set up JDK 17
        uses: actions/setup-java@v2
        with:
          java-version: '17'
          distribution: 'temurin'
      - name: Build and Test with Maven
        run: mvn clean test
```

2. Commit e push para o GitHub:

```
bash

git add .
git commit -m "Adiciona pipeline CI"
git push origin main
```

3. Verificar a execução no GitHub:

- Acesse `Actions` no repositório para ver o pipeline rodando.

Exercício Prático para os Alunos (20 min)

Desafio:

1. Modifique o projeto para incluir um novo método `subtract()` na classe `Calculator`.
2. Adicione um teste JUnit para esse método.
3. Faça um push e verifique se o CI passa.

Código de Exemplo:

```
java

// Novo método em Calculator.java
public static int subtract(int a, int b) {
    return a - b;
}

java
```

```
// Novo teste em CalculatorTest.java
@Test
public void testSubtract() {
    assertEquals(1, Calculator.subtract(3, 2));
}
```

Encerramento

- **Discussão:** Como CI ajuda a manter a qualidade?
- **Próxima Aula:** Padrões de codificação e boas práticas (Clean Code, Checkstyle).

Tarefa Opcional:

- Pesquisar sobre "GitHub Actions para deploy automático".

Recursos:

- [GitHub Actions Docs](#)
- [Scrum Guide](#)

Pronto para a próxima aula? Qualquer dúvida, é só perguntar! 😊🚀